GachaDesk

基于 AMD Radeon GPU (ROCm) 的多模态内容创作工具

Track 1 项目简介文档

团队：xihuyisheng

应用：GachaDesk

硬件：AMD Instinct MI308X (192GB HBM3)

1. 项目背景

GachaDesk 是一款基于 AMD Radeon GPU (ROCm) 生态构建的多模态内容创作工具，整合了文生图、图生图编辑、图生视频三大能力，通过统一的 Gradio 网页工作台，让用户在浏览器中即可端到端完成分镜驱动的短视频制作。

生成式 AI 正在重塑内容创作，但现有工具普遍存在三大痛点：

- 工作流碎片化：文生图、图生图、图生视频需要在多个独立工具间切换。
- 硬件锁定：开源推理管线几乎全部针对 CUDA 优化，AMD ROCm 用户缺乏成熟方案。
- 部署门槛高：20B+ 参数的扩散模型对显存和工程能力要求极高。

GachaDesk 将三个最先进的开源模型部署在 AMD Instinct MI308X（192GB HBM3）上，通过统一的瘦客户端界面对外提供服务。用户即使在低配终端上，也能完成专业级视觉内容创作，真正实现算力与交互的解耦。

2. 目标用户与应用场景

目标用户

- 乡村教师与教育工作者，需要为课堂快速生成配图素材。
- 中小企业与直播带货商家，低成本制作商品展示图与宣传短视频。
- 文旅与农产品营销人员，批量产出景区意境图与分镜短片。
- 内容创作者与短视频制作人，批量产出多分镜画面。
- ROCm 开发者，评估 AMD MI300 系列 GPU 上真实的多模态推理性能。

应用场景

工作台支持完整的「分镜脚本到成片视频」工作流：

- 乡村助教课件 —— 为乡村教师快速生成配图素材，让课件生动直观。
- 女装宣传展示 —— 一键生成服装上身效果图与宣传短视频，低成本打造商品展示。
- 文旅风光推广 —— 生成景区意境图与分镜短片，助力乡村文旅宣传。
- 农产直播带货 —— 为农产品制作精美海报与预告短片，提升带货转化。
- 短视频素材库 —— 批量产出多分镜画面，快速合成创意短视频。

分镜工作流

工作台提供 8 个分镜槽位（2 行 x 4 列）。用户上传或生成种子图后，推入分镜 #1，为每个分镜生成视频，并通过「续帧」功能自动提取上一个视频的最后一帧作为下一个分镜的输入，确保视觉连贯性。最终 8 段视频可合并为一条长视频，并支持添加水印。

3. 系统架构

GachaDesk 采用瘦客户端 + 远程 GPU 服务的部署架构。主项目自身不加载任何模型，而是通过 ModelScope 创空间提供的 Gradio API，调用三个独立部署的子服务。

主项目（瘦客户端）

位置：src/app.py。可在任意环境（本地 / 边缘 / 低配机器）运行，负责用户交互、分镜编排、视频合片与水印绘制。零 GPU 依赖，仅依赖 requests、gradio、imageio、PIL。

三个远程子服务（重 GPU 端）

每个子服务独立部署在 AMD MI308X 上，通过 ModelScope 创空间暴露 Gradio API + SSE 流式响应：

- Qwen-Image-2512-AMD-MI308X —— 文生图（20B MMDiT 扩散模型）
- Qwen-Image-Edit-2511-AMD-MI308X —— 图生图编辑（背景替换、风格转换、人物换装）
- Wan2.2-14B-I2V-LoRA-AMD-MI308X —— 图生视频（14B DiT + 蒸馏 LoRA）

API 调用流程

主项目通过 ModelScopeImageClient 封装完整的远程调用流程：

- GET /config —— 探测 API 端点与 api_prefix
- POST /gradio_api/call/<api_name> —— 提交任务，获得 event_id
- GET /gradio_api/call/<api_name>/<event_id> —— 监听 SSE 流获取结果
- GET /gradio_api/file=... —— 通过 API 网关鉴权下载产物
- POST /gradio_api/upload —— 上传待编辑图片

鉴权使用 Bearer Token（从 .env 的 MODELSCOPE_API_TOKEN 读取）。

算力解耦模型

这种架构让算力与交互解耦：真正的 ROCm 推理算力集中在云端 MI308X 子服务，主项目可在任何低配终端运行，适合乡村助教、直播带货等终端资源有限的场景。

4. 模型与算法介绍

4.1 Qwen-Image-2512（文生图）

通义千问 20B 参数 MMDiT（多模态扩散 Transformer）模型。接收中英双语文本提示词，自动追加质量后缀（超清 4K、电影级构图），支持 7 种官方推荐分辨率（1:1 / 16:9 / 9:16 / 4:3 / 3:4 / 3:2 / 2:3）。内置负向提示词，规避低画质、肢体畸形、AI 感等问题。

4.2 Qwen-Image-Edit-2511（图生图）

指令驱动的图像编辑 MMDiT 扩散模型，支持背景替换、风格转换、人物换装等编辑任务。挂载 lightx2v/Qwen-Image-Edit-2511-Lightning 蒸馏 LoRA，4 步即可出图。输入图片中心裁剪至目标宽高比，保持主体不变形。

4.3 Wan2.2-I2V-A14B（图生视频）

Wan-AI 14B 参数图生视频 DiT 模型。给定参考图和文本提示词生成短视频。挂载 lightx2v/Wan2.2-Distill-Loras 的 high_noise + low_noise 双适配器（各 50% 权重），4 步极速推理。固定 16 FPS，支持 1280x720 / 832x480 / 768x432 三档分辨率，81 或 121 帧。

4.4 分镜合片算法

瘦客户端本地实现两种合片模式：

- 8 图合片：8 张分镜图片按 16 帧/张拼接为幻灯片视频（8 FPS）。
- 8 镜合片：8 段分镜视频顺序拼接为一条长视频，保留首段视频的 FPS。

两种模式均支持水印渲染：每帧右下角绘制黑阴影白文字标签，字号自适应帧宽。

4.5 续帧机制

为确保分镜间视觉连贯性，续帧功能提取视频 N 的最后一帧，作为分镜 N+1 的输入图，无需手动截图即可创建从镜头 1 到镜头 8 的连贯视觉链条。

5. AMD Radeon GPU / ROCm 适配说明

三个子模块均运行在单张 AMD Instinct MI308X（192GB HBM3）卡上，基于 ROCm 6.3.0+ 与 PyTorch ROCm 版本进行本地推理。各模块针对自身模型特性分别采用了不同的 ROCm 优化组合。

5.1 Qwen-Image-2512 优化

- 精度：bfloat16（ROCm 原生支持）
- 注意力：flash_attention_2 + preferred_rocm_fa_library('aotriton') 后端
- 矩阵乘法：allow_tf32=True + set_float32_matmul_precision('high')，MI300+ TF32 加速显著
- 显存：全量 GPU 加载，20B MMDiT 完整载入 192GB HBM3，零换页
- 容错：VAE float16 解码失败自动回退 float32，关键节点 torch.cuda.synchronize() 避免异步错误

5.2 Qwen-Image-Edit-2511 优化

- 精度：bfloat16
- 注意力：enable_flash_sdp(True) + enable_mem_efficient_sdp(True)，禁用 math_sdp，强制 Triton Flash Attention
- 关键环境变量：TORCH_ROCM_AOTRITON_ENABLE_EXPERIMENTAL=1、FLASH_ATTENTION_TRITON_AMD_ENABLE=TRUE
- 关键环境变量：HIP_FORCE_DEV_KERNARG=1、PYTORCH_HIP_ALLOC_CONF=garbage_collection_threshold:0.8,max_split_size_mb:512
- 缓存持久化：TRITON_CACHE_DIR / TORCH_EXTENSIONS_DIR 重定向至 /mnt/workspace，避免容器重启后重编译（首次编译约 20 分钟，后续秒出）
- 加速：lightx2v Lightning 蒸馏 LoRA，4 步出图
- 分辨率锁定：固定 1664x928，避免不同 shape 触发内核重编译

5.3 Wan2.2-I2V 优化

- 精度：float16
- 注意力：xformers（失败自动回退默认实现）
- 加速：high_noise + low_noise 双蒸馏 LoRA（各 50% 权重），4 步推理
- 输出：固定 16 FPS，1280x720 / 832x480 / 768x432，81 或 121 帧
- 监控：集成 rocm-smi 实时获取温度、核心/显存频率

5.4 共性优化

- 本地全量推理：模型完整载入 MI308X 显存，不走量化分片或 CPU offload
- Gradio + MCP：mcp_server=True，每个 API 端点映射为 MCP 工具，支持智能体调用
- 日志双通道：logger.info / print 同时输出到控制台与 Gradio 界面运行日志框
- 显存治理：每次推理前后 gc.collect() + torch.cuda.empty_cache()，避免碎片累积

--- 文档结束 ---